```
GET programming_books1/_search
{
  "query": {
    "multi_match": {
      "query": "Elasticsearch hands-on example driven",
      "fields": ["title","synopsis"]
    }
  }
}
```

Searches for the document using a multi_match query

Invokes a multi_match query to search across

The fields to search

This query searches for the criteria across multiple fields. It is an expensive query, based on the number of fields and documents (imagine searching over a dozen fields in numerous books). We can put a combined field pattern into action using the `copy_to` attribute on individual fields to alleviate such problems.

The following listing creates a mapping for the `programming_books2` index. The notable thing is the addition of a separate field called `title_synopsis`, which is a `text` data type.

Listing 15.2 Enhanced schema definition with a `copy_to` attribute

```
PUT programming_books2
{
  "mappings": {
    "properties": {
      "title":{
        "type": "text",
        "copy_to": "title_synopsis"
      },
      "synopsis":{
        "type": "text",
        "copy_to": "title_synopsis"
      },
      "title_synopsis":{
        "type": "text"
      }
    }
  }
}
```

Copies the title info to title_synopsis

Copies the synopsis info to title_synopsis

Denotes title_synopsis as a text field

As you can gather from the schema definition in this listing, each of the two fields has an additional `copy_to` attribute with a value pointing to a third field, `title_synopsis`. Any data indexed into the `title` and `synopsis` fields is copied into `title_synopsis` behind the scenes. The next listing shows the indexing of a sample book document into the `programming_books2` index.

Listing 15.3 Indexing a sample document

```
PUT programming_books2/_doc/1
{
  "title":"Elasticsearch in Action",
  "synopsis":"A straightforward, hands-on, example-driven, action book"
}
```